



UNIVERSIDAD DE PANAMÁ
CENTRO REGIONAL UNIVERSITARIO DE COCLÉ
“DOCTOR BERNARDO LOMBARDO”
FACULTAD DE INFORMÁTICA, ELECTRÓNICA Y COMUNICACIONES
LICENCIATURA DE INGENIERÍA EN INFORMÁTICA

ASIGNATURA:
INFORMÁTICA TEÓRICA

TEMA:
TEORIA DE CONJUNTOS

PROFESORA:
RICHARD DOMINGUEZ

ESTUDIANTES:
IAN SOTILLO
DAYAN PÉREZ

2026

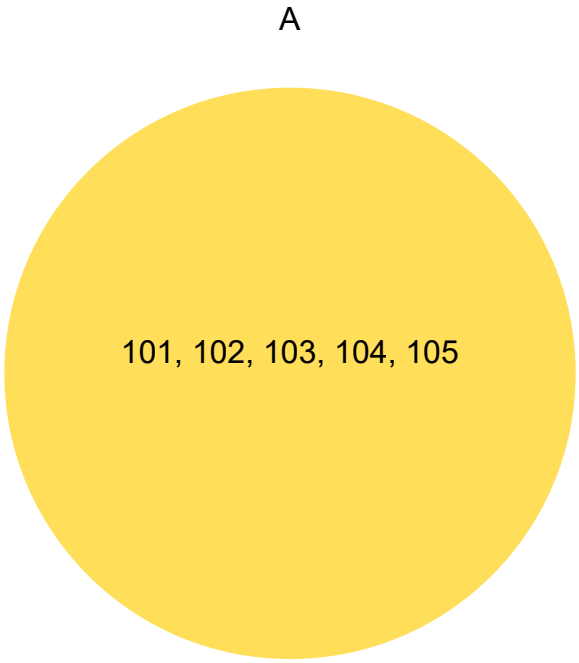
INTRODUCCIÓN

La informática teórica establece los cimientos lógicos y formales que sustentan el procesamiento de datos en la ciencia de la computación moderna. Dentro de este marco, la teoría de conjuntos funge como un modelo matemático indispensable para estructurar, auditar y manipular colecciones de información mediante relaciones de pertenencia, inclusión y operaciones algebraicas. El presente trabajo tiene como objetivo central representar gráficamente y comprobar algorítmicamente dieciséis conceptos y operaciones fundamentales de conjuntos, utilizando como caso de estudio bases de datos reales correspondientes a registros académicos en formato JSON (estudiantes_programacion.json y estudiantes_redes.json).A lo largo de esta investigación práctica, se aborda metódicamente la preparación y depuración de los datos eliminando redundancias y unificando criterios de unicidad para construir conjuntos representativos (A y B) dentro de un universo definido (U). Mediante el diseño de diagramas de Venn analíticos y su respectiva ejecución y validación programática en Python, se exploran exhaustivamente propiedades estructurales que abarcan desde la cardinalidad y los subconjuntos propios hasta estructuras avanzadas como los frozenset y la comprobación formal de las leyes lógicas (conmutatividad, asociatividad, idempotencia y distributividad) mediante asertos computacionales. Este estudio no solo demuestra la aplicabilidad directa de los modelos abstractos en la gestión y el cruce de bases de datos heterogéneas, sino que también evidencia el impacto técnico de las restricciones de hardware y tipado en el desarrollo de software robusto.

TEMA #1
Concepto de Conjunto

Un conjunto es una colección bien definida de objetos distintos (elementos) donde el orden de aparición no altera su identidad estructural.

Diagrama de Venn



Representación Matemática:

A = { x | x es ID unico de estudiante en programacion }

Código Python:

```
set_prog = set([101, 102, 101, 103])
set_prog_orden = set([103, 101, 102])
print("Set con duplicados:", set_prog)
print("¿Conservan igualdad al cambiar orden?:", set_prog == set_prog_orden)
```

Resultado:

Conjunto A = {101, 102, 103, 104, 105}

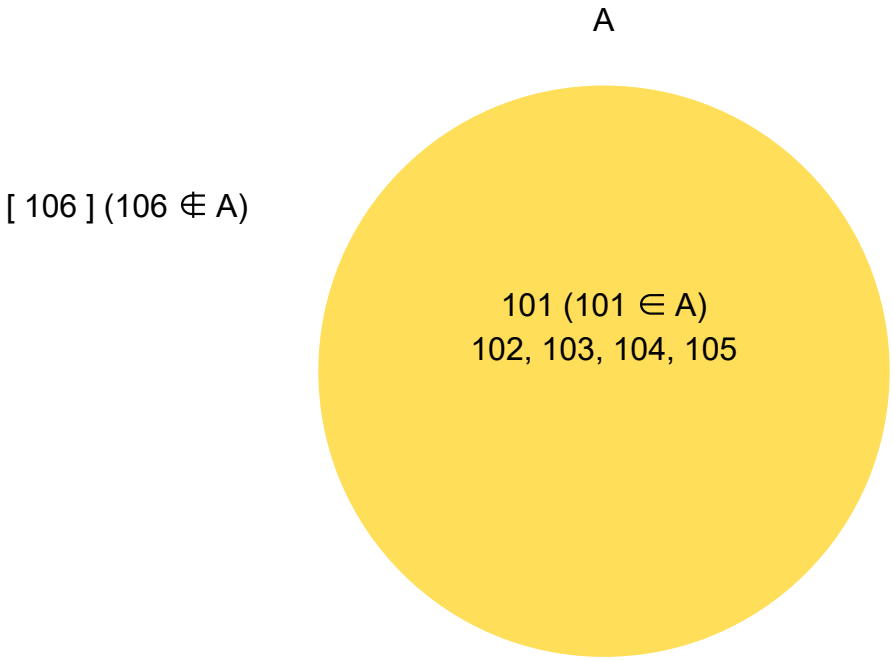
Explicación

: Los archivos fuente contienen duplicados. Al estructurarlo como conjunto en Python, la propiedad de unicidad elimina automáticamente las repeticiones.

TEMA #2
Notación y Pertenencia

Relación fundamental que indica si un elemento específico forma parte de un conjunto (E) o no (∉)

Diagrama de Venn



Representación Matemática:

101 esta en A (pertenece) Y 106 no esta en A (no pertenece)

Código Python:

```
print("101 in A:", 101 in A)
print("106 not in A:", 106 not in A)
```

Resultado:

True para ambas evaluaciones.

Explicación

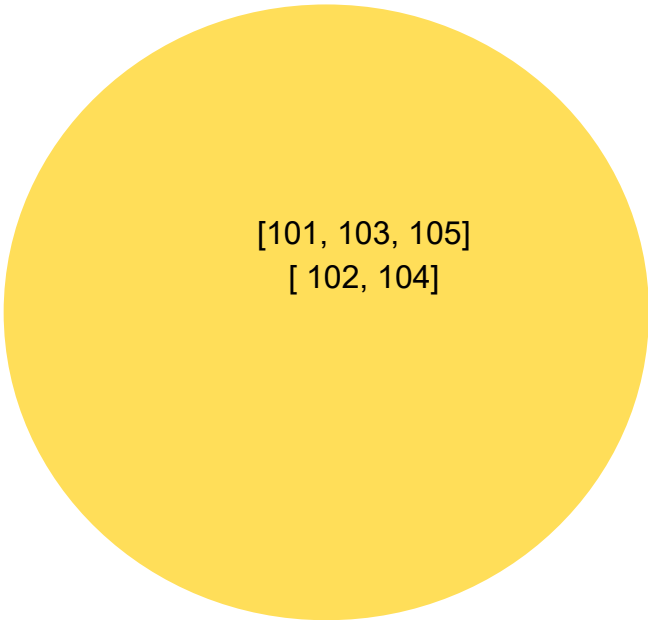
: Valida analíticamente que el ID 101 pertenece a Programación (A), mientras que el ID 106 está excluido.

TEMA #3
Extensión y comprensión

Formas de definir conjuntos por enumeración explícita o por predicado lógico. /

Diagrama de Venn

A



Representación Matemática:

Extension: A = { 101, 102, 103, 104, 105 }
Comprension: C = { x que pertenece a A tal que x mod 2 == 0 }

Código Python:

```
extensión = {101, 102, 103, 104, 105}
comprensión = {x for x in A if x % 2 == 0}
print("Extensión:", extensión)
print("Comprensión (IDs pares de A):", comprensión)
```

Resultado:

Resultado: $C = \{102, 104\}$.

Explicación

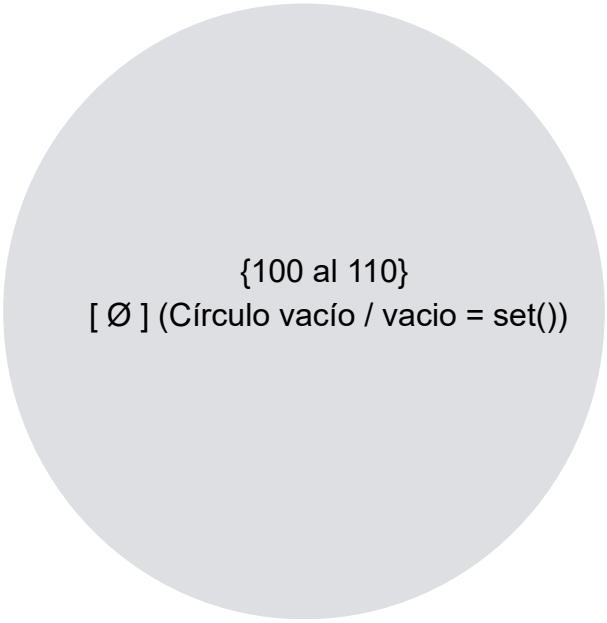
: La comprensión filtra dinámicamente registros en Python extrayendo solo los que cumplen con la condición de paridad.

TEMA #4
Conjunto Vacío y Universo

Formas de definir conjuntos por enumeración explícita o por predicado lógico. /

Diagrama de Venn

U



Representación Matemática:

Conjunto Vacio (vacio) = { }
Cardinalidad de vacio = 0 Universo (U) = { 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110 }

Código Python:

```
vacio = set()
print("len(vacio):", len(vacio))
print("type(set()):", type(set()), "| type({}):", type({}))
```

Resultado:

Longitud 0, tipo set.

Explicación

: Inicializar un conjunto vacío requiere set(), ya que {} crea un diccionario vacío en Python.

TEMA #5
Conjuntos Numéricos y Limitación de Python

representan los diferentes tipos de números que se pueden utilizar en operaciones matemáticas y cálculos dentro de un programa.

Diagrama de Venn



Representación Matemática:

IDs pertenecen a los Naturales (N) subset de Enteros (Z) subset de Racionales (Q) subset de Reales (R)

Código Python:

```
print("IDs en N:", all(isinstance(x, int) and x >= 0 for x in A))
print("Limitación de float en R (0.1 + 0.2):", 0.1 + 0.2)
```

Resultado:

0.30000000000000004.

Explicación

: Los IDs pertenecen a los naturales (N). El tipo float en R tiene precisión finita por restricciones de almacenamiento binario.

TEMA #6
Subconjuntos y Subconjuntos Propios

Es un conjunto cuyos elementos están contenidos dentro de otro conjunto. /

Diagrama de Venn

A



Representación Matemática:

C subseteq A (C es subconjunto de A) C subseet A (C es subconjunto propio de A, donde C != A)

Código Python:

```
C = {x for x in A if x <= 103}
print("C < A (Subconjunto propio):", C < A)
print("A <= A (Subconjunto):", A <= A)
print("A < A (Subconjunto propio de sí mismo):", A < A)
```

Resultado:

True para ambas condiciones.

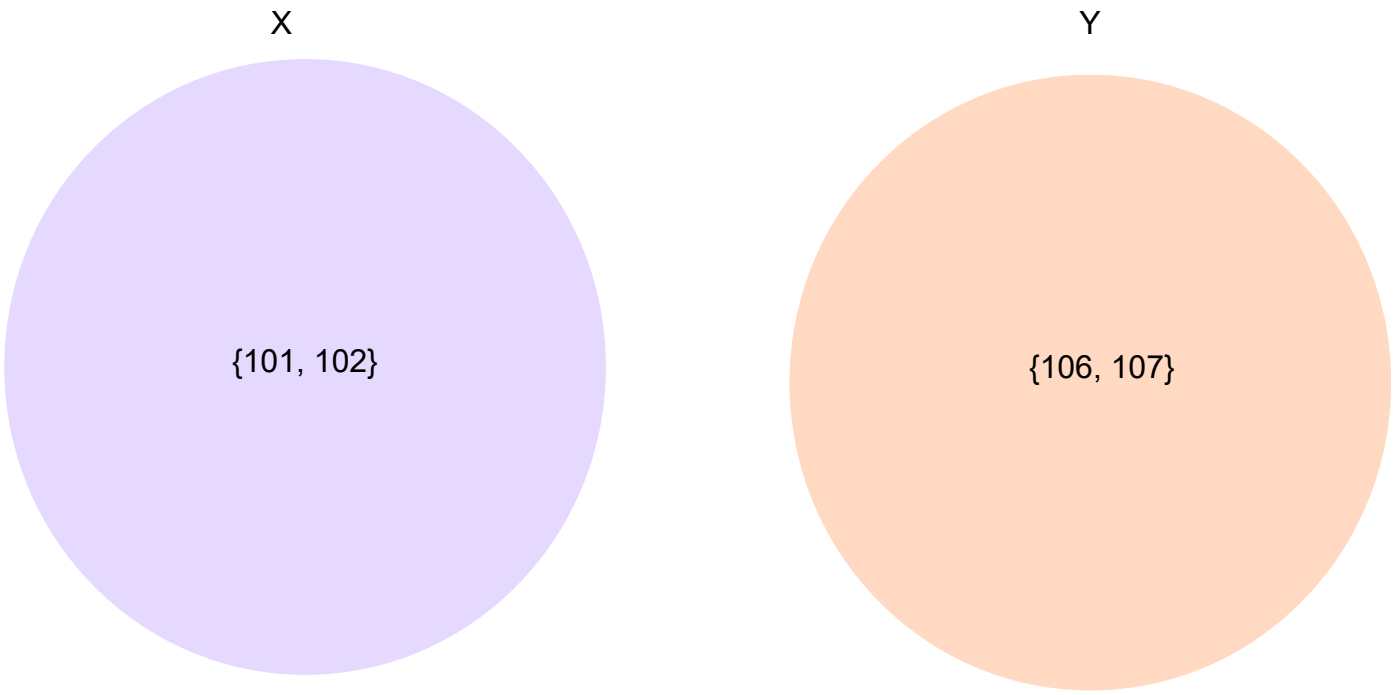
Explicación

: Un subconjunto propio exige que todos los elementos de C estén en A siendo C más pequeño. Por reflexividad, todo conjunto es subconjunto impropio de sí mismo (A C A).

TEMA #7
Igualdad y Conjuntos Disjuntos

La igualdad de conjuntos ocurre cuando dos conjuntos contienen exactamente los mismos elementos, sin importar el orden en que se encuentren.

Diagrama de Venn



Representación Matemática:

Igualdad: A == B (si tienen exactamente los mismos elementos) Disjuntos: X interseccion Y == vacio (no comparten elementos)

Código Python:

```
print("A == B:", A == B)
print("A.isdisjoint(B):", A.isdisjoint(B))
X = {101, 102}
Y = {106, 107}
print("X.isdisjoint(Y):", X.isdisjoint(Y))
```

Resultado:

False para igualdad de A y B; True para la disyunción de X e Y.

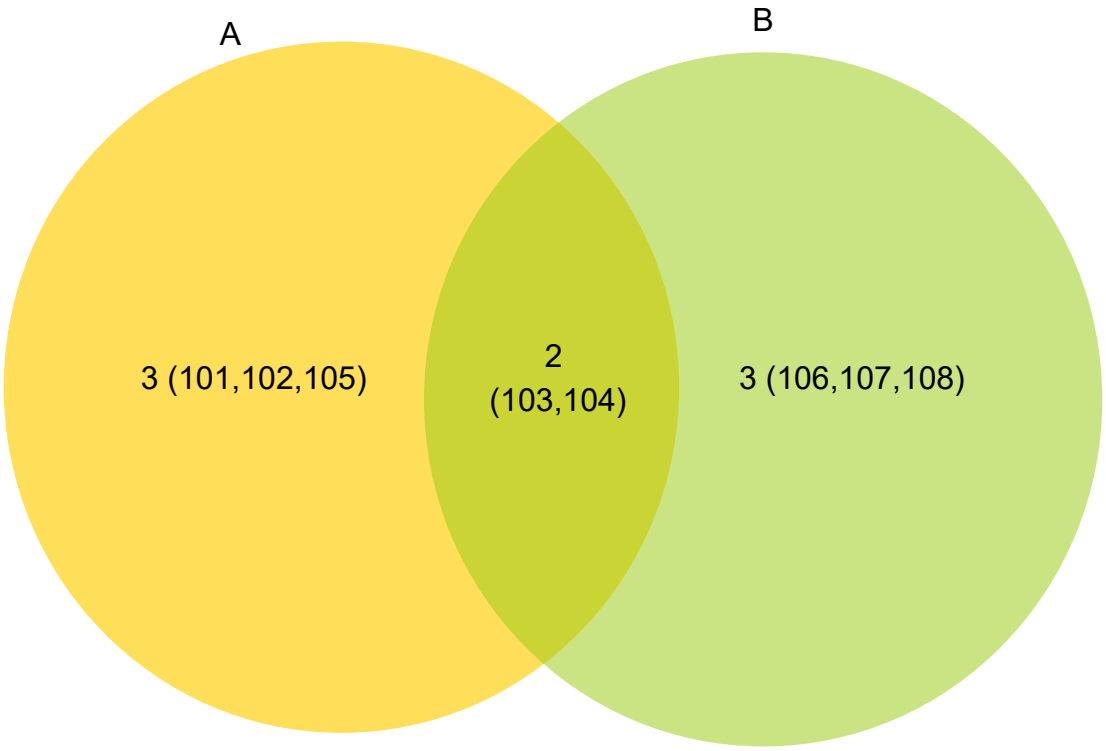
Explicación

: Permite evaluar si las listas contienen los mismos registros o si corresponden a universos totalmente separados.

TEMA #8
Cardinalidad de Conjuntos Finitos

es la cantidad de elementos diferentes que contiene un conjunto /

Diagrama de Venn



Representación Matemática:

|A| = 5, |B| = 5, |A interseccion B| = 2

Código Python:

```
print("len(A):", len(A))
print("len(B):", len(B))
print("len(A & B):", len(A & B))
```

Resultado:

FResultado: |A| = 5, |B| = 5

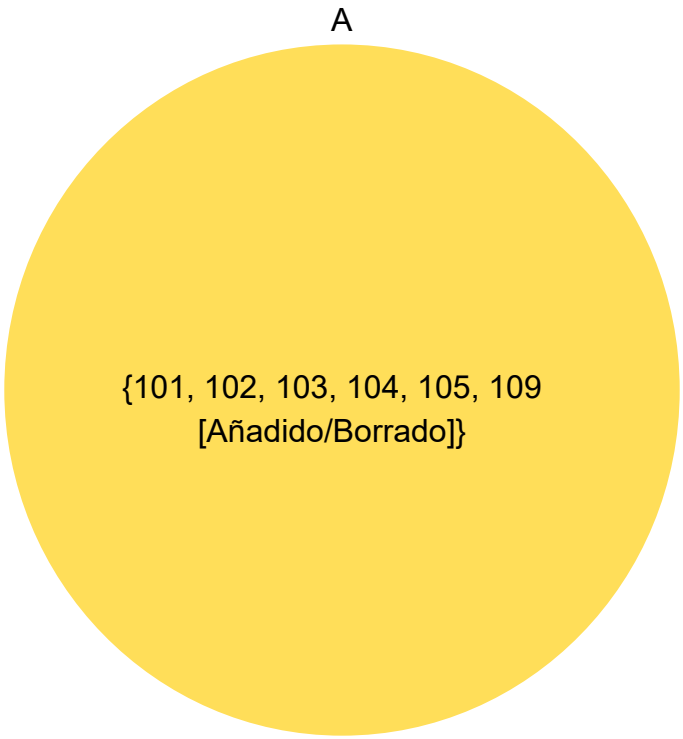
Explicación

: Aunque los archivos tenían 6 registros planos, la cardinalidad matemática elimina duplicados reflejando el número real de individuos únicos.

TEMA #9
Creación y Modificación de Conjuntos

consiste en definir colecciones de elementos únicos y realizar cambios en su contenido mediánte diferentes métodos.

Diagrama de Venn



Representación Matemática:

A_nuevo = A union { 109 }
A_final = A_nuevo menos { 109 }

Código Python:

```
A_copia = A.copy()
A_copia.add(109)
print("Tras add(109):", A_copia)
A_copia.discard(109)
print("Tras discard(109):", A_copia)
A_copia.discard(999)
try:
    A_copia.remove(999)
except KeyError:
    print("remove(999) arrojó KeyError correctamente")
```

Resultado:

Ejecución segura sin errores mediante discard().

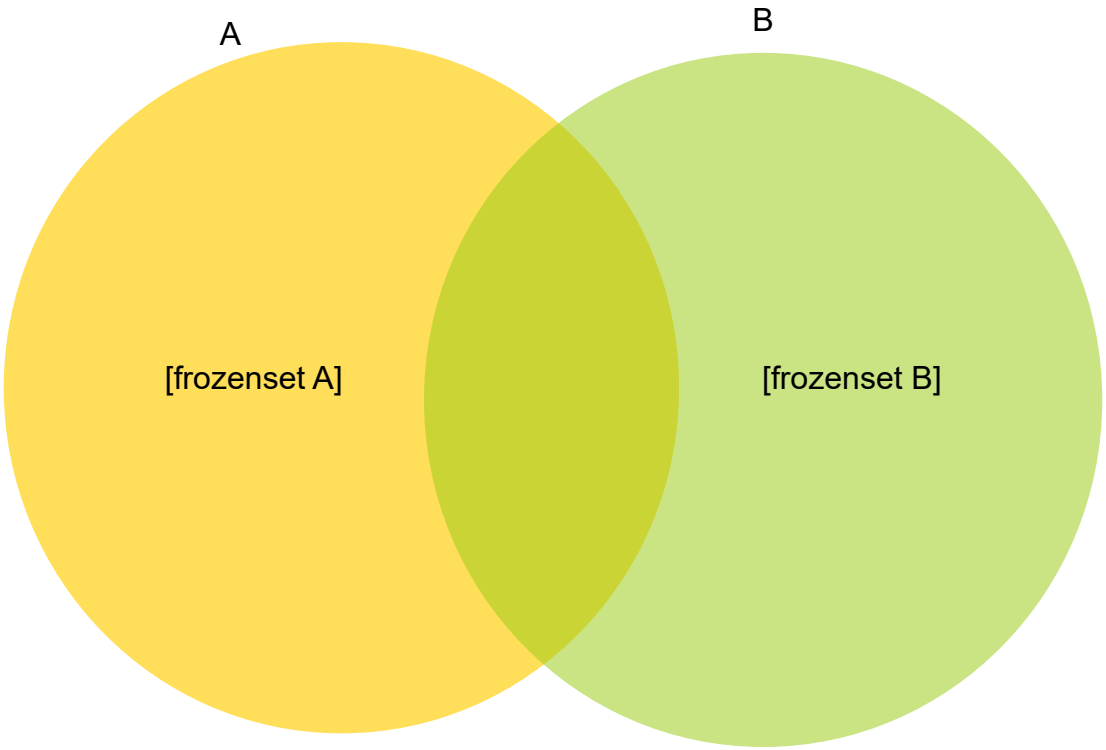
Explicación

: discard() elimina un elemento sin arrojar errores si no existe, a diferencia de remove(), que lanza una excepción KeyError.

TEMA #10
Elementos Hashables y Frozenset

son objetos que tienen un valor hash que no cambia durante su existencia y que pueden compararse con otros objetos.

Diagrama de Venn



Representación Matemática:

F = { frozenset(A), frozenset(B) }

Código Python:

```
f_a = frozenset(A)
f_b = frozenset(B)
conjunto_de_conjuntos = {f_a, f_b}
print("Conjunto con frozensets:", conjunto_de_conjuntos)
```

Resultado:

Creación exitosa de conjuntos de conjuntos.

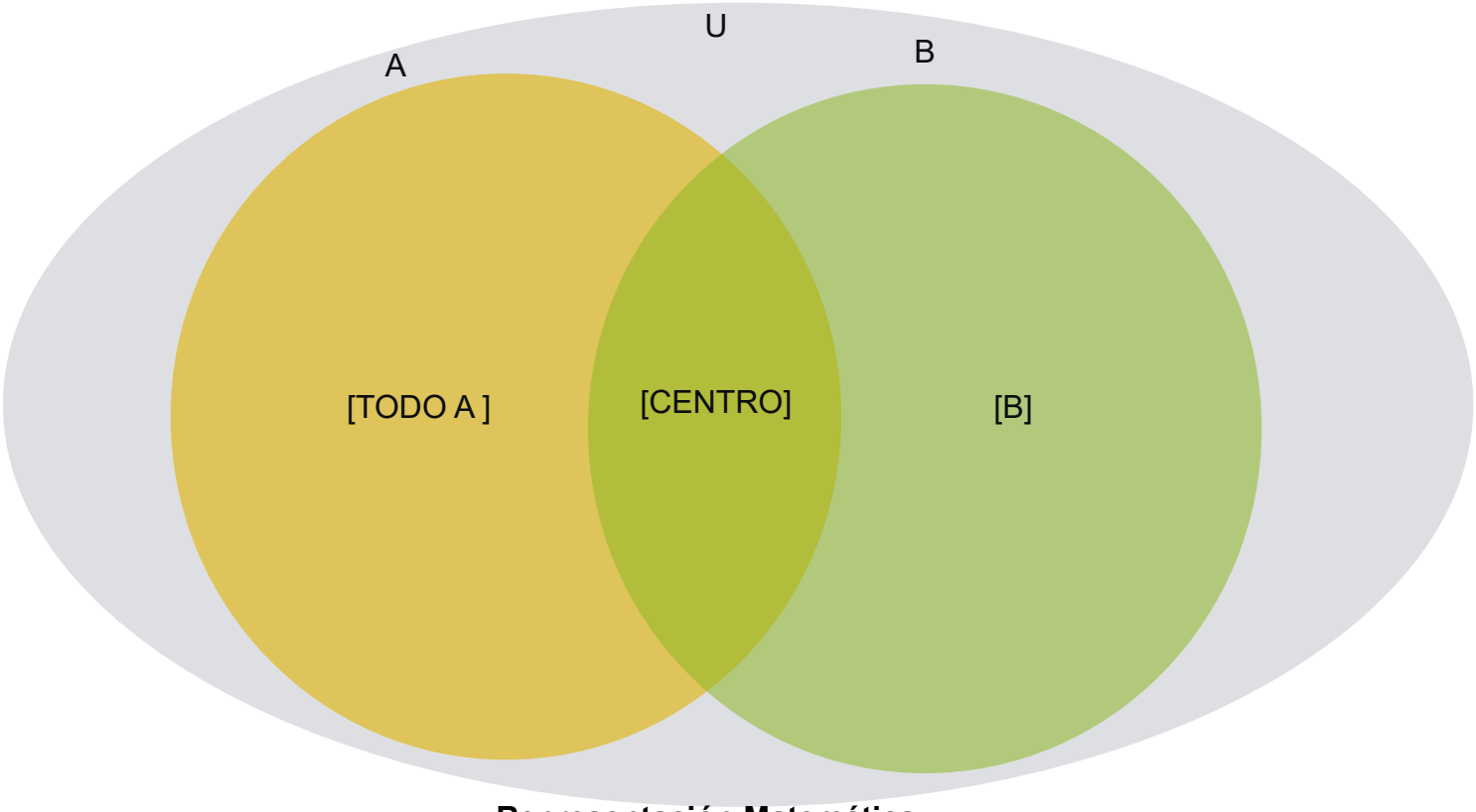
Explicación

: Las listas y sets estándar son mutables (no hashables). El frozenset resuelve esto al ser inmutable y poseer un hash válido.

TEMA #11
Unión (A U B)

Es una operación matemática que consiste en combinar todos los elementos de dos o más conjuntos, sin repetir aquellos que aparecen en ambos.

Diagrama de Venn



Representación Matemática:

A union B = { x | x pertenece a A o x pertenece a B }

Código Python:

```
print("A | B:", A | B)
print("Igual a A.union(B):", (A | B) == A.union(B))
```

Resultado:

{101, 102, 103, 104, 105, 106, 107, 108\}

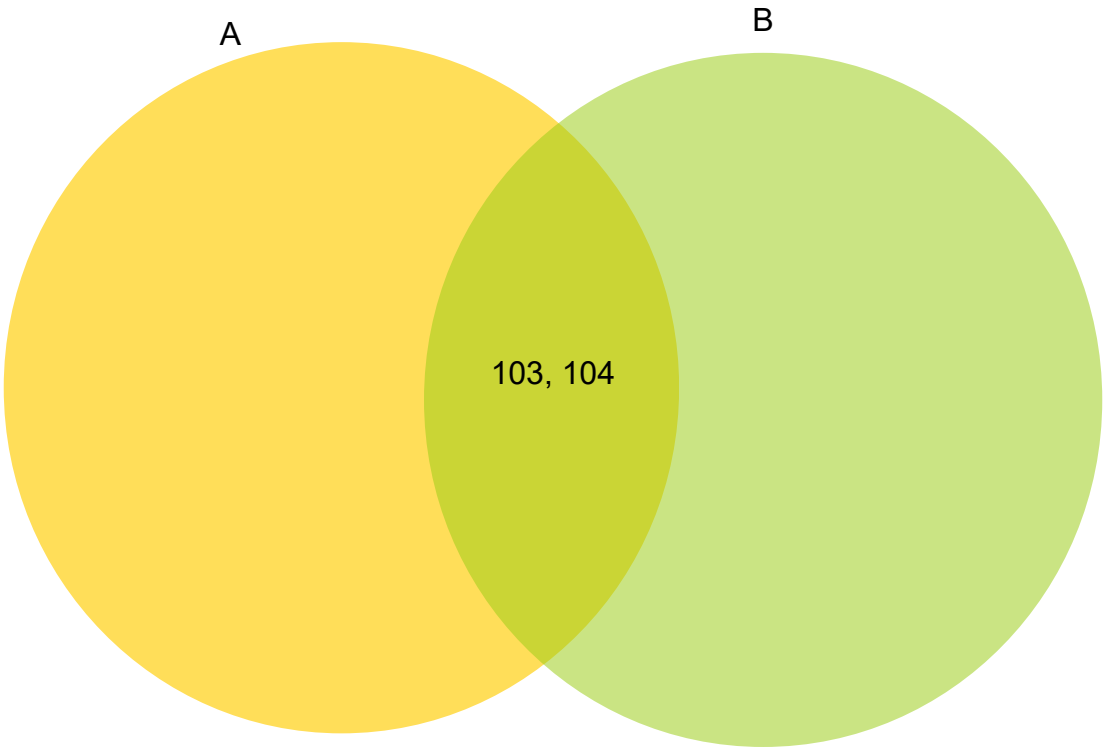
Explicación

: Agrega la matrícula total de estudiantes únicos que cursan al menos una de las dos asignaturas.

TEMA #12
Intersección (A ∩ B)

Es una operación matemática que permite obtener los elementos que tienen en común dos o más conjuntos.

Diagrama de Venn



Representación Matemática:

A interseccion B = { x | x pertenece a A y x pertenece a B }

Código Python:

```
print("A & B:", A & B)
print("Igual a A.intersection(B):", (A & B) == A.intersection(B))
```

Resultado:

{103, 104}

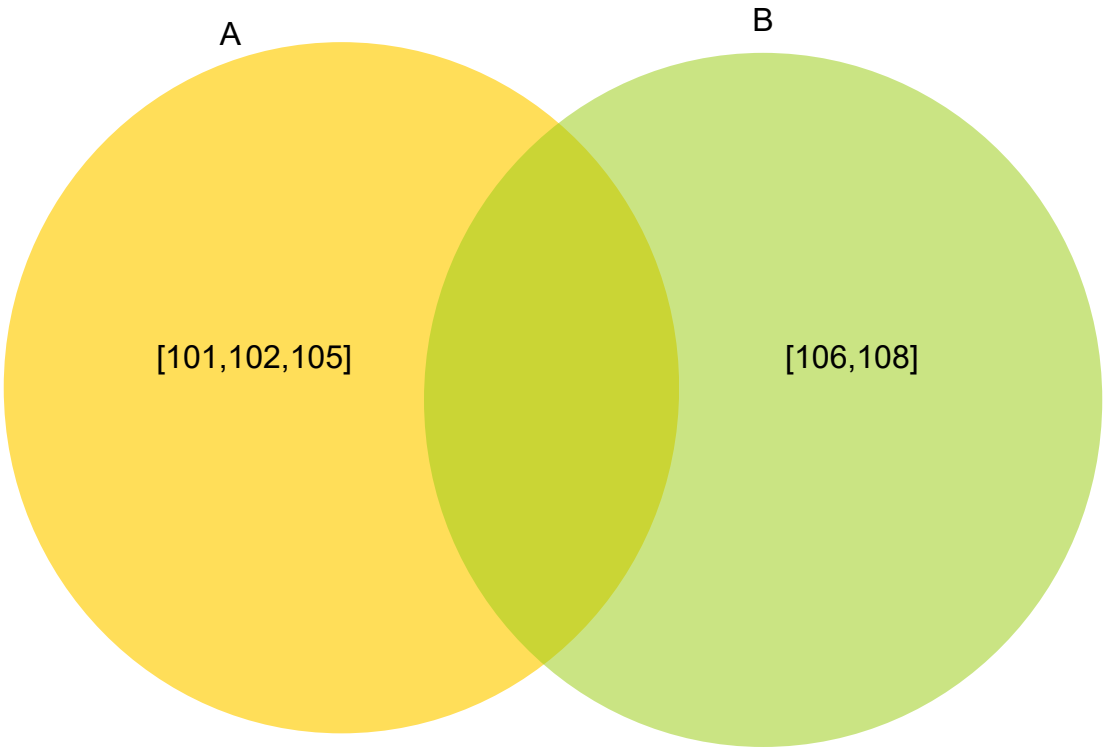
Explicación

: Identifica al grupo de estudiantes que cursan ambas materias simultáneamente.

TEMA #13
Diferencia de Conjuntos (A − B y B − A)

Es una operación matemática que permite obtener los elementos que pertenecen a un conjunto, pero que no se encuentran en el otro.

Diagrama de Venn



Representación Matemática:

A menos B = { x | x pertenece a A y x no pertenece a B } B menos A = { x | x pertenece a B y x no pertenece a A }

Código Python:

```
print("A - B:", A - B)
print("B - A:", B - A)
```

Resultado:

A - B = {101, 102, 105} | B - A = {106, 107, 108}

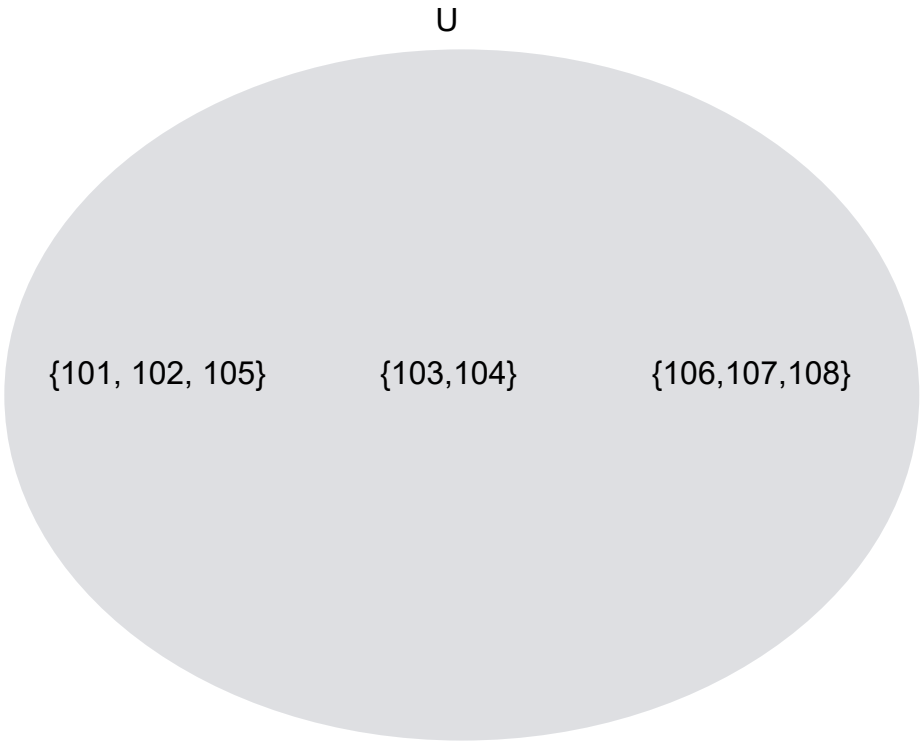
Explicación

: Separa los estudiantes exclusivos de Programación frente a los exclusivos de Redes.

TEMA #14
Diferencia Simétrica (A▲B)

Es una operación matemática que permite obtener los elementos que pertenecen a uno de los dos conjuntos, pero no a ambos al mismo tiempo.

Diagrama de Venn



Representación Matemática:

A triangulo B = (A menos B) union (B menos A)

Código Python:

```
print("A ^ B:", A ^ B)
print("Equivale a (A - B) | (B - A):", (A ^ B) == ((A - B) | (B - A)))
```

Resultado:

{101, 102, 105, 106, 107, 108}

Explicación

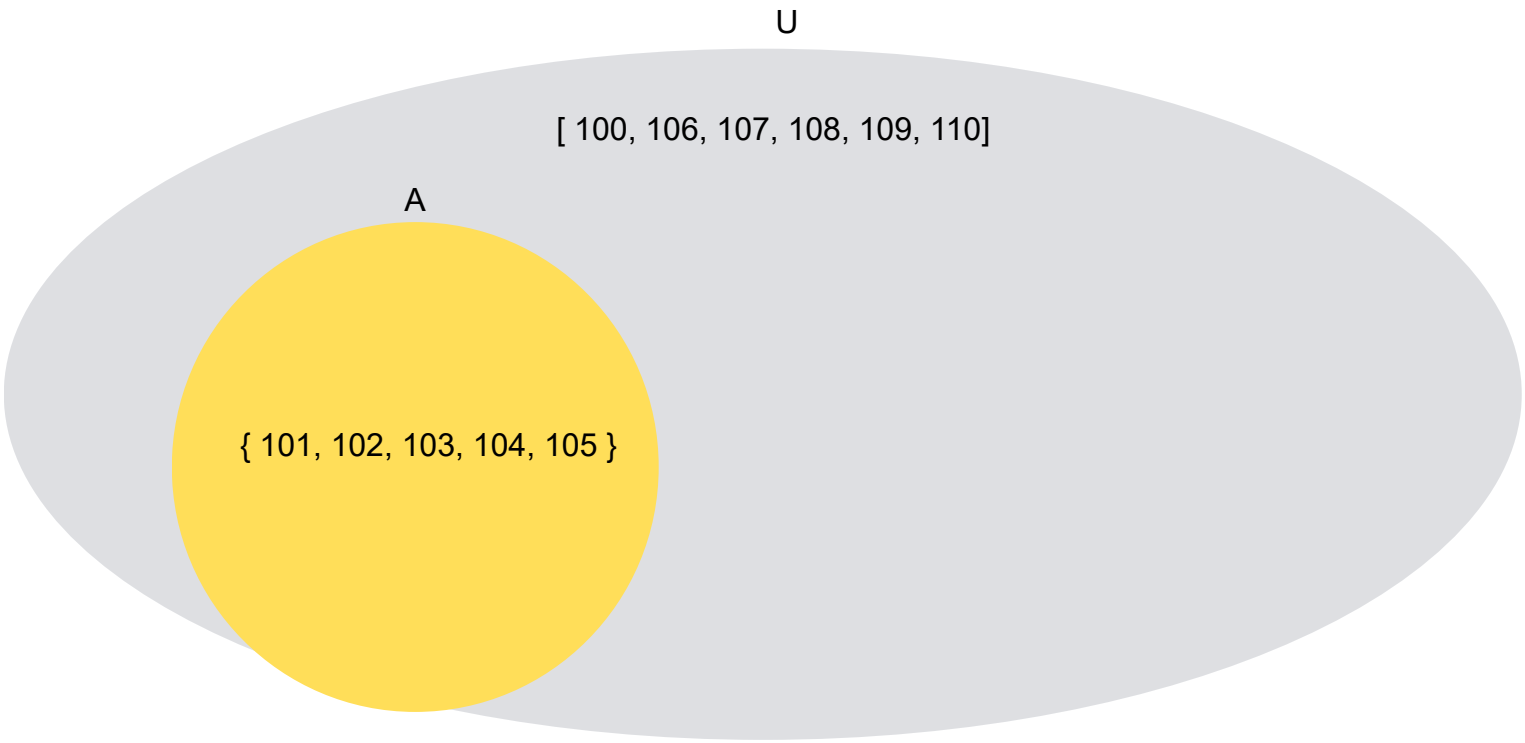
: Estudiantes que cursan una sola de las materias, descartando a los matriculados en ambas.

TEMA #15

Complemento (A') ▲

Es una operación matemática que permite obtener todos los elementos que pertenecen al conjunto universal (U), pero que no están presentes en el conjunto (A).

Diagrama de Venn



Representación Matemática:

Complemento de A = U menos A = { x que pertenece a U tal que x no pertenece a A }

Código Python:

```
print("A <= U:", A <= U)
complemento_A = U - A
print("Complemento U - A:", complemento_A)
```

Resultado:

{100, 106, 107, 108, 109, 110}

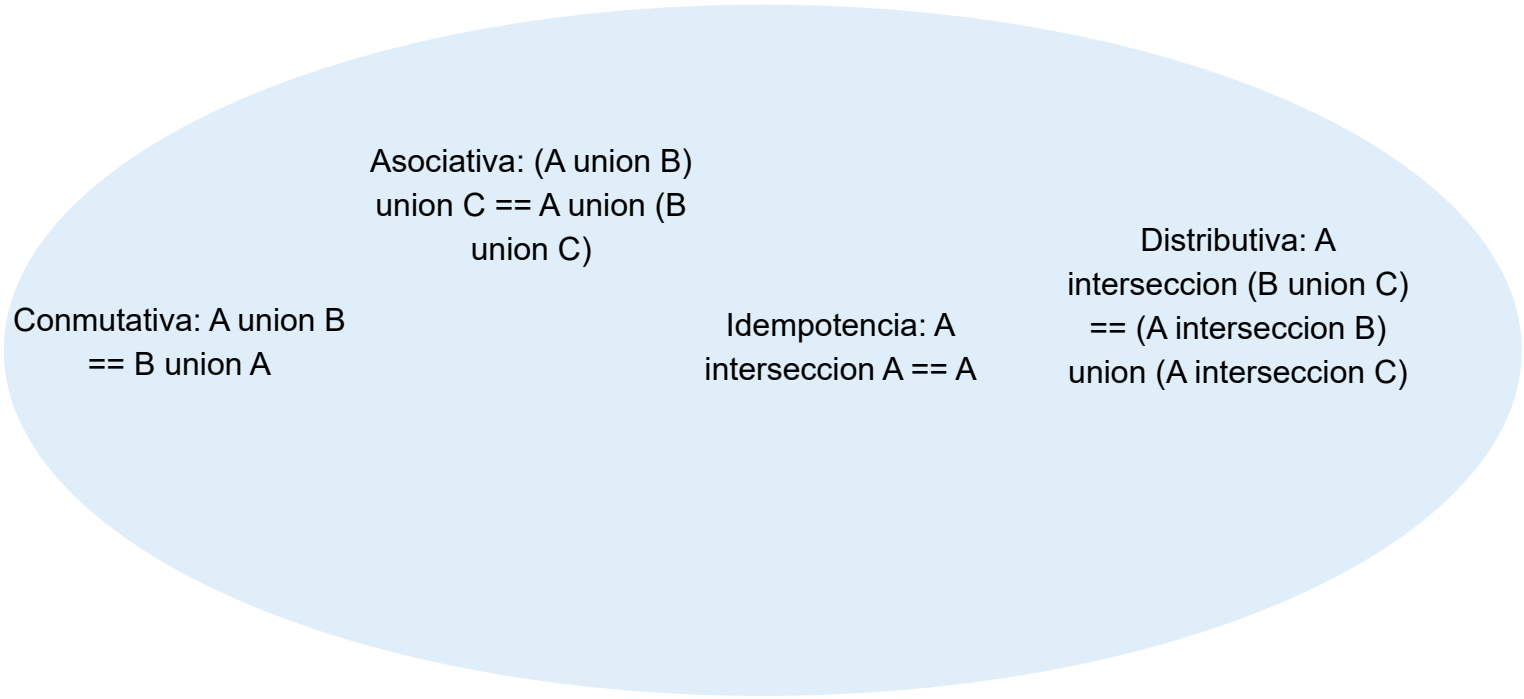
Explicación

: Demuestra que el complemento abarca a los estudiantes de la facultad que no cursan Programación.

TEMA #16
Propiedades de las Operaciones y Conclusión

cumplen diversas propiedades matemáticas que permiten realizar cálculos, comparar datos y simplificar instrucciones dentro de un programa.

Diagrama de Venn



Representación Matemática:

- Conmutativa: A union B == B union A
- Asociativa: (A union B) union C == A union (B union C)
- Idempotencia: A interseccion A == A
- Distributiva: A interseccion (B union C) == (A interseccion B) union (A interseccion C)

Código Python:

```
C_prop = {x for x in U if x >= 105}
assert (A | B) == (B | A)
assert ((A | B) | C_prop) == (A | (B | C_prop))
assert (A & A) == A
assert (A & (B | C_prop)) == ((A & B) | (A & C_prop))
print("¡Todas las propiedades comprobadas exitosamente con assert!")
```

Resultado:

Comprobación exitosa mediante asertos lógicos (assert).

Explicación

: Las operaciones de unión, intersección y diferencia simétrica demostraron ser las herramientas más robustas para procesar y cruzar los registros de estudiantes, permitiendo auditar la unicidad de las matrículas de forma rigurosa

Conclusión Grupal

El desarrollo exhaustivo de esta práctica académica ha permitido establecer un puente metodológico sólido y directo entre el rigor abstracto de la teoría formal de conjuntos y su implementación práctica en entornos de programación modernos mediante Python. A lo largo del análisis de los archivos de datos estructurados (estudiantes_programacion.json y estudiantes_redes.json), se constató que la recolección de información en escenarios reales suele presentar anomalías estructurales, tales como registros redundantes y duplicidades que distorsionan el tamaño poblacional aparente. En este sentido, la aplicación estricta del concepto de conjunto que exige la unicidad de sus elementos mediante la propiedad de membresía demostró ser un mecanismo de depuración y auditoría sumamente eficaz, permitiendo transformar colecciones planas de datos en estructuras matemáticas limpias.

Asimismo, el uso sistemático de las operaciones fundamentales aportó perspectivas analíticas clave para el procesamiento de la información. La unión facilitó la consolidación de la matrícula total sin solapamientos, mientras que la intersección se reveló como la herramienta indispensable para aislar con exactitud matemática a aquellos individuos que cursan múltiples asignaturas de forma simultánea. Por su parte, la diferencia y la diferencia simétrica permitieron seccionar y segmentar los grupos exclusivos y heterogéneos, optimizando la capacidad de extracción de subconjuntos de interés analítico sin alterar la integridad de los datos originales.

Desde la perspectiva de la arquitectura de datos y la programación orientada a objetos en Python, la exploración de tipados avanzados como la distinción fundamental entre diccionarios {} y conjuntos vacíos set(), la gestión controlada de excepciones con KeyError, y la implementación de colecciones inmutables mediante frozenset evidenció que las restricciones teóricas de la informática (tales como la hashabilidad y la precisión finita de los números reales representados en tipos de datos flotantes) impactan directamente en el diseño de software robusto. Finalmente, la comprobación formal de las leyes lógicas del álgebra de conjuntos (conmutatividad, asociatividad, idempotencia y distributividad) mediante la validación programática con asertos (assert) reconfirmó que la formalización matemática no constituye un mero ejercicio teórico abstracto, sino un pilar fundamental para garantizar la consistencia, la previsibilidad lógica y la optimización algorítmica en el procesamiento moderno de bases de datos.